Lab Exercises

Steps to execute

1. **Download, unzip, and paste the unzipped folder to the base location**

2. **Run Docker, while mounting the local filesystem folder:**

   docker run --rm -it -v .\ca\week6_gem5_labs\:/work -w /work amansinhaatnycu/ca:latest bash

3. **Compile:**

   ./compile_riscv.sh

4. **Run:**

   ./run_lab1.sh

   ./run_lab2.sh

   ./run_lab3.sh

Lab Exercise 1: Branch Pattern Sensitivity

- **What is the code asking?**
  - Does the branch behavior pattern affect performance even under the same predictor?
  - Which branch style is easier for the processor to handle?
    - Loop-like
    - Alternating
    - Correlated
- **What does the code do?**
  - Runs the same program in three branch modes:
    - loop
    - alt
    - corr
  - Each mode generates a different branch-outcome pattern
  - Measures:
    - Instruction count
    - Cycle count
    - IPC

Questions

1. Which mode has the lowest numCycles and highest IPC?
2. Is the loop-style branch easier than the alternating branch?
3. Does the correlated pattern behave more like a loop or alternating?

Lab Exercise 2: Predictable vs Unpredictable Branches

- **What is the code asking?**
  - How much does branch predictability affect performance?
  - What happens when the processor speculates on:
    - Mostly predictable control flow
    - Hard-to-predict control flow

- **What does the code do?**
  - Runs the program in two modes:
    - pred
    - unpred
  - Keeps the overall structure similar
  - Changes how easy the branch stream is to predict
  - Measures:
    - Simulated time
    - Cycle count
    - IPC
  - Shows the performance cost of poor speculation

Questions

1. How much does IPC drop from pred to unpred?
2. Do simInsts stay similar while numCycles rises?
3. What does that imply about the misprediction penalty and wrong-path work?

Lab Exercise 3: Direct vs Indirect Control Flow

- **What is the code asking?**
  - Is target complexity a performance problem, not just branch direction?
  - Does the processor handle:
    - Direct control flow
    - Indirect control flow

    equally well?

- **What does the code do?**
  - Runs the program in two modes:
    - direct
    - indirect
  - Uses control flow with:
    - simpler fixed targets in one case
    - more complex target behavior in the other
  - Measures:
    - Cycle count
    - IPC
    - Simulated time

Questions

1. Does direct outperform indirect?
2. Why can target complexity matter even when there is still only one control transfer at a time?
3. How does this connect to BTB/RAS/indirect-target prediction from the lecture?